P.Abdul Reg No:192525426

SIMATS ENGINEERING

Department of Computer Science and Engineering (CSE)

ITA0507 – Computer Vision

ASSIGNMENT 1 – CO1

CO1: Apply the fundamental concepts, image formation models, and processing techniques for analyzing Computer Vision systems.

Student Name	M. Pavan Kumar	**Register Number**	192411117
Department / Section	CSE	**Academic Year / Semester**	2025–2026 / _____
Faculty Name	Murali P	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	____ . ____ . _____	**Date of Evaluation**	____ . ____ . _____

Assignment Title

Apply, Compare and Justify Image Acquisition, Formation and Processing Techniques for a Vision-Based E-Waste Battery Sorting Line (Total Marks – 50)

MARK SHEET (Rubric-based Evaluation)

Task	Focus	Max Marks	Marks Awarded
Task 1	Overview and Features of Computer Vision: Systems Comparison	4	
Task 2	Levels of Computer Vision and Image Processing: Pipeline Depth	5	
Task 3	Digital Image Fundamentals I: Colour Representations	4	
Task 4	Digital Image Fundamentals II: Pixel Connectivity	4	
Task 5	Image Sensing and Acquisition Systems	5	
Task 6	Sampling and Spatial Limits	5	
Task 7	Quantization and Intensity Limits	4	
Task 8	Image Formation Models (Geometric / Photometric)	5	
Task 9	Applications and Integrated Edge Systems	6	
Task 10	Re-apply and Adapt (Engineering Scaling Challenge)	8	
	TOTAL	50	

Scenario and Derived Design Parameters

GreenCycle Robotics Corp. sorts mixed consumer batteries on a bulk dump belt. Operators tire, miss swellings and dents, and cannot read small laser-etched chemistry codes under changing light. The plant needs a camera system over the belt that (1) classifies form factor, (2) reads chemistry codes, and (3) detects safety defects, then triggers a pneumatic piston kick-rejecter. As the vision engineer, I apply three candidate techniques in every task, compare them, and choose one with evidence.

The operating constraints given in the question were converted into the design quantities below. Each derived value is used in the tasks that follow.

Table 1: Derived design parameters

Quantity	Calculation	Value
Belt speed	given	1.2 m/s = 1200 mm/s
Decision deadline	piston 0.48 m downstream ÷ 1.2 m/s	0.48 / 1.2 = 0.40 s = 400 ms
Required resolution for 0.3 mm crack (Nyquist, 2 px)	0.3 mm ÷ 2 px	≤ 0.15 mm/px (about 2133 px across 320 mm)
Preferred resolution (3 px per 0.3 mm)	0.3 mm ÷ 3 px	0.10 mm/px
Minimum frame rate (cell 70 mm in 240 mm FOV)	1200 ÷ (240 − 70)	7.06 fps → use 8 fps, one trigger per cell
Cells per second, 5-day archive	15 cells/s × 86 400 s × 5 days	6 480 000 cells
Pouch swelling precision	given	±0.5 mm

Experimental Setup Used for All Tasks

Every quantitative result in this document was produced by one Python program (OpenCV 4.13, NumPy 2.4, scikit-learn 1.8) using a fixed random seed, so each run gives the same numbers. Real battery photographs were not available, so the experiments use generated images in which the true defect size, depth and position are known. This lets me change one factor at a time (light, blur, resolution, bit depth). The limitation is that generated images cannot reproduce every real effect, such as real rust texture, foil reflections or dirt. Every result must therefore be repeated on real cells before commissioning. The sample sizes are small (60 cells, 12 pairs, 10 cells per row), so differences of a few per cent are not significant.

Table 2: Experimental setup

Item	What was used
Target compute	4-core edge PC, 16 GB RAM, no GPU; Python + OpenCV + NumPy (times below are measured on the development machine, the same language stack)
Test cells	Generated top-down views of a battery on a black belt; length 45–70 mm, 30 mm wide, cylindrical and flat shading, rust spots (0.6 mm, 35 levels deep), shallow dents (4 mm, 5–9 levels deep), glare, cast shadow
Camera model	Full HD grid 1920 × 1080 at 0.1667 mm/px unless stated; motion blur = v × t; rolling shutter = row-wise shift with 30 ms readout and 100 Hz flicker
Human graders (Task 1)	Simulated with stated miss rates; labelled as simulated where used
Repeatability	Fixed seeds for every experiment

TASK 1 – Systems Comparison: Manual, Assisted and Automated Sorting [4 Marks]

Question: compare manual inspection (A), image-processing-assisted sorting (B) and fully automated computer vision (C) on processing speed, repeatability, objectivity (Cohen's kappa), non-contact safety, endurance, traceability and cost, and critique each against the 15 cells/s line rate.

Table 3: Three sorting workflows

Technique	Basic idea
A. Manual inspection	Human sorters under shed ambient light decide form, chemistry and defects by eye
B. Image-processing-assisted	High-resolution camera feeds contrast-enhanced, zoomed frames to operator screens; the operator still decides
C. Automated computer vision	Camera plus rule-based software classify cells and fire the rejection piston without human input

Ans: The line must handle 15 cells/s and commit a decision within 400 ms. I simulated three human sorters (G1–G3), each grading the same 60 cells twice: round 1 at the start of the shift and round 2 at the end, when fatigue adds misses. The automated grader C was run on the same cells. The human miss rates are assumed values (45 % for shallow dents, 15 % for rust spots, a 6 % false-reject rate for clean cells) and are labelled as assumptions, not measurements.

Table 4: Comparison of the three workflows (simulated where stated)

Criterion	A. Manual	B. Assisted	C. Automated CV
Speed (cells/s)	≈ 0.5 per sorter (2 s per cell, assumed)	same as A: a human still decides	≈ 19 cells/s per core on a full 1080p frame (median 54 ms)
Sorters needed for 15 cells/s	≈ 30 (plus breaks)	≈ 30	1 PC (4 cores)
Repeatability (intra-rater kappa)	0.53 (simulated)	better than A, still human	1.00 (same input → same output)
Objectivity (inter-rater kappa)	0.43 (simulated)	not measured	same rule for every cell
Accuracy vs truth (60 cells)	82.8 % round 1; 75.0 % round 2	not measured	75.0 %, kappa vs truth 0.50
Non-contact, endurance	handling by hand; fatigue by end of shift	non-contact; fatigue remains	non-contact, continuous 24 × 7
Traceability	paper sheet only	images can be saved	image, measurements and decision logged for every cell
Cost	recurring wages for ≈ 30 sorters	monitors + staff	one-time camera, lens, light and PC (see Task 9)

Figures in Table 4 are from the simulation; the human rows are assumptions and must be replaced with the real sorters' sheets before any claim is made about people.

Measured finding: the automated grader found 15 of the 30 defective cells, but missed all 15 cells with shallow dents. Its accuracy (75.0 %) is therefore below the simulated human round 1 (82.8 %) on this test set. This is the main limitation of C in its present form and it is carried into Tasks 9 and 10.

Differences in the four related fields: image processing takes an image and returns an image (e.g. contrast enhancement in B); computer vision takes an image and returns a description or decision (cell class, defect present, in C); machine vision adds hardware to act on the decision (camera, strobe, encoder trigger, PLC and piston); computer graphics runs the other way, generating an image from a description. I used generated images, which is computer graphics, to test the vision system.

Final Decision. I choose C (automated computer vision). It is the only option that can meet 15 cells/s with one machine, and its output is repeatable and traceable. Its limit is that it misses shallow dents, so it cannot be commissioned until a shading-corrected dent detector has been added and tested on real cells. Until then, every cell in the buyer-audit window must be reviewed by a trained inspector, which is what the plant needs to meet the 1.5 % miss limit.

TASK 2 – Pipeline Level Depth Comparison [5 Marks]

Question: compare how deep the software must go: low-level only (A), low plus mid-level (B), and low plus mid plus high-level with a classifier (C). Tabulate accuracy and latency on a test set with hard cases, explain how shadows trigger false alarms in low-level processing, contrast with CNN/ViT learning, and give a level-annotated pipeline.

Table 5: Three pipeline depths

Technique	Operations
A. Low level only	Gaussian blur, adaptive threshold; count of dark pixels inside the cell flags a defect (count threshold tuned on the training set)
B. Low + mid level	Adds contours, minimum-area rectangle, aspect ratio, dark-spot components; static expert rules
C. Low + mid + high level	Mid-level feature vector into an RBF support vector machine (scikit-learn SVC) that outputs defect class

Test set: 60 ordinary cells (40 clean, 20 defective) and 20 hard cases (shadow bands, glare and an oil smudge that looks like a defect). Training set: 120 cells. Latency is the median time per cell crop (80 × 40 mm at 0.1667 mm/px).

Table 6: Accuracy and latency per pipeline depth

Technique	Accuracy, all (%)	Accuracy, ordinary (%)	Accuracy, hard (%)	Missed defects (FN)	False rejects (FP)	Latency (ms, crop)
A. Low only	80.0	83.3	70.0	14	2	2.10
B. Low + mid	81.2	83.3	75.0	15	0	2.41
C. Low + mid + high	75.0	80.0	60.0	5	15	3.60

Shadow effect: on a clean cell with a shadow band, the low-level count rose to 568 dark pixels; without the shadow it was 0. The shadow edge is a sharp intensity step, and an adaptive threshold reads it as a dark boundary, which is why low-level arrays cannot separate a shadow from a defect.

Reading Table 6: all three depths miss many defects. The shallow dents are the largest source of error for the rule-based grader, which found none of the 15 dents in the Task 1 test. Depth alone does not solve the problem: B and C use the same features, and C's SVM gives more false rejects on the hard cases. The latency of all three is far below the 400 ms budget; full-frame mid-level processing measured 54 ms median on a 1080p frame.

Contrast with CNN/ViT: an end-to-end network learns low-level filters (edges, colour), mid-level parts (rust specks, seams) and high-level classes inside its layers, without hand-written thresholds. It needs thousands of labelled images per class for training from scratch, or a few hundred per class with a pretrained network. The classical pipeline needs only a validation set of about 50–100 cells, so the trade-off is accuracy on varied real cells (learning) against explainability and low data needs (classical).

Figure: level-annotated pipeline (LOW = pixels to pixels, MID = pixels to attributes, HIGH = attributes to decision)

Stage	OpenCV operation	Level	Input → output
1. Acquisition + trigger	encoder trigger, strobe	machine vision	belt → raw frame

2. Denoise	cv2.medianBlur, GaussianBlur	LOW	image → image
3. Foreground mask	Otsu threshold, morphology close	LOW	image → binary mask
4. Cell region	cv2.connectedComponentsWithStats	MID	mask → one region
5. Shape and size	cv2.findContours, cv2.minAreaRect	MID	region → length, width, aspect
6. Defect candidates	closing, dark-component labelling	MID	image → spot list (area, shape)
7. Decision	rules (B) or SVM (C)	HIGH	feature vector → class + confidence
8. Actuation	PLC output to piston	machine vision	class → rejection signal

Final Decision. I choose the three-level pipeline with rules (Technique B) for the first release. B reached 81.2 % overall with no false rejects, and it is explainable to the factory and the buyer. C scored lower on the hard cases, and A misses the mid-level features it needs. The classifier in C can be added later, once a labelled archive exists. Shadows and touching cells are limits of all three and must be handled by the mid level (watershed splitting, shadow-robust segmentation) before C can be trusted.

TASK 3 – Colour Representation for Rust Analysis [4 Marks]

Question: identify the most stable colour representation for brownish-red iron-oxide rust on steel battery jackets, comparing RGB (A), HSV hue with S and V for highlights (B), and CIE L*a*b* with ΔE distance from a healthy steel reference (C). Quantify performance at 1.0×, 0.4× and 1.8× light, and prove why NumPy addition can overflow.

Method: each cell was rendered with a brownish-red rust spot (0.6 mm, 35 levels darker, with a red-high, blue-low tint). For each representation I computed a per-pixel score (A: $|R - B|$; B: saturation × rust-hue gate, i.e. $S \cdot [H \leq 25$ or $H \geq 160]$; C: ΔE from a fixed healthy-steel Lab reference measured at baseline light). Each cell gets the maximum of the smoothed score inside the cell. The threshold is set once at the midpoint of the baseline means, then the same threshold is used at all light levels (the factory sets it once and light changes).

Table 7: Rust detection accuracy (%) across light levels, 60 cells (30 with rust)

Representation	Dim 0.4×	Baseline 1.0×	Intense 1.8×	Miss at dim (%)	Behaviour
A. RGB ($\|R - B\|$)	50	100	100	100	the colour difference scales with light, so the fixed threshold fails when light drops
B. HSV (hue + saturation)	100	100	100	0	hue and saturation do not depend on brightness, so the score is unchanged
C. CIE L*a*b* (ΔE)	50	100	50	0	ΔE from a fixed reference is stable in dim light, but at 1.8× the casing saturates and its L* moves away from the reference

Dispersion: the mean score of rust cells against healthy cells at baseline was 33.0 vs 0.0 for RGB, 66.9 vs 0.0 for HSV, and 28.3 vs 2.6 for Lab. For HSV, rust cells score 67 against 0 for healthy cells, a clear gap that does not depend on light.

Array architecture and integer overflow

An 8-bit colour image is a NumPy array of shape (rows M, columns N, channels 3) with dtype uint8. A pixel is img[y, x] (row first), so the pixel at column 100, row 20 is img[20, 100]. The storage for one uncompressed Full HD frame is $b = M \times N \times k \times \text{channels} = 1080 \times 1920 \times 8 \times 3 = 49\,766\,400$ bits ≈ 6.2 MB, and for one 80 × 40 mm crop at 0.1667 mm/px it is 240 × 480 × 8 × 3 bits ≈ 0.35 MB.

Overflow. For uint8, NumPy wraps around modulo 256: 250 + 10 = 260 mod 256 = 4. OpenCV's cv2.add saturates at 255: min(250 + 10, 255) = 255. A brightness boost written as img + 10 in NumPy therefore turns a bright peel pixel almost black. The correct form is cv2.add(img, 10) (or np.clip on a wider type). In this line, a 10-level brightness correction on a glossy casing would make the highlight disappear and create false dark regions.

Final Decision. I choose B (HSV, using hue and saturation). It was 100 % correct at all three light levels, while RGB fell to 50 % in dim light and Lab to 50 % in intense light. The rust-hue gate also protects against the glossy

highlight, which has low saturation. Lab remains useful when a buyer asks for a colour-difference value (ΔE), but it needs its reference re-measured for each light level.

TASK 4 – Pixel Connectivity for Casing Scratch Counting [4 Marks]

Question: the quality system rejects a cell with more than 2 surface scratches or any rust spot above 0.5 mm equivalent diameter. Compare 4-connectivity (A), 8-connectivity (B) and m-connectivity (C), test them on synthetic patterns, and prove which distance measure (De, D4, D8) gives true physical size.

Method: I wrote my own m-connectivity labelling (two-pass with union-find, diagonal link allowed only when both shared 4-neighbours are background) because OpenCV provides only 4 and 8. Each rule was run on four synthetic patterns of known truth and on a rendered cell with a scratch and a spot.

Table 8: Object counts on synthetic patterns (true count known)

Pattern	True	4-conn	8-conn	m-conn
diagonal 1-px line (1 object)	1	12	1	1
checkerboard joint (2 touching blobs at corners)	1	2	1	1
two spots touching at a corner	2	2	1	1
two spots + 3% salt noise	2	22	20	20

Rendered cell with a 1-px diagonal scratch and one rust spot: 4-connectivity produced 71 pieces, which would be counted as scratches and would reject good cells. 8-connectivity kept the scratch and the spot in 1 object, which is correct for the scratch but merges the spot into it, so the spot size would be overstated. The count for a single line, 12 pieces under 4-connectivity against 1 under 8-connectivity, shows why 4-connectivity is unsuitable for diagonal scratches.

Behaviour: m-connectivity gives the same object counts as 8-connectivity on all four patterns. It only removes redundant diagonal links (double paths), which matters for tracing boundaries, not for counting. Under 3 % salt noise, all rules count far too many objects (20–22 against 2 true), so a minimum area filter (3 px) and a median pre-filter are required; these were applied in Task 2 and are part of the recommended grader.

Table 9: Distance measures for a rust spot at 45° (analytical, 0.1667 mm/px)

True diameter (mm)	De (Euclidean)	D4 (city-block)	D8 (chessboard)
0.3	0.30	0.42	0.21
0.5	0.50	0.71	0.35
1.0	1.00	1.41	0.71

For a disk of diameter d measured along the 45° diagonal, the opposite edge points are at (a, a) and (−a, −a) with $a = d/(2\sqrt{2})$. The three measures are then De = d, $D4 = \sqrt{2}\cdot d \approx 1.41\ d$, and $D8 = d/\sqrt{2} \approx 0.71\ d$. D4 over-estimates a diagonal spot by 41 %, so a 0.4 mm spot would be rejected as larger than 0.5 mm. D8 under-estimates by 29 %, so a 0.7 mm spot would be passed. Only De is independent of direction, and so it is the correct measure for the 0.5 mm rule after calibration (mm = pixels × mm/px).

Final Decision. I choose 8-connectivity with a 3-pixel minimum-area filter for counting, and the Euclidean measure De for size. 4-connectivity breaks diagonal scratches into many false pieces. m-connectivity gives the same count as 8-connectivity; my implementation is pure Python and slower than OpenCV, so its advantage is only for boundary tracing. Touching spots that merge side by side need a distance-transform watershed; no connectivity rule can split them.

TASK 5 – Sensor Shutter and Exposure Topology [5 Marks]

Question: compare area-scan rolling shutter with constant light (A), global shutter with an LED strobe synchronized to the encoder (B), and line-scan (C) at 1.2 m/s. Calculate blur for 4.0, 1.0 and 0.05 ms, quantify rolling-shutter distortion with 100 Hz flicker, map the signal chain, and justify CCD or CMOS.

Table 10: Motion blur at 1.2 m/s (Full HD, 0.1667 mm/px)

Exposure	Blur = v × t (mm)	Blur (px at 0.1667 mm/px)	Verdict for a 0.3 mm crack
4.0 ms	4.80	28.8	fails: the crack is 1.8 px wide and is smeared over 28.8 px

1.0 ms	1.20	7.2	fails: smears over 7.2 px
0.05 ms	0.06	0.36	passes: blur below half a pixel

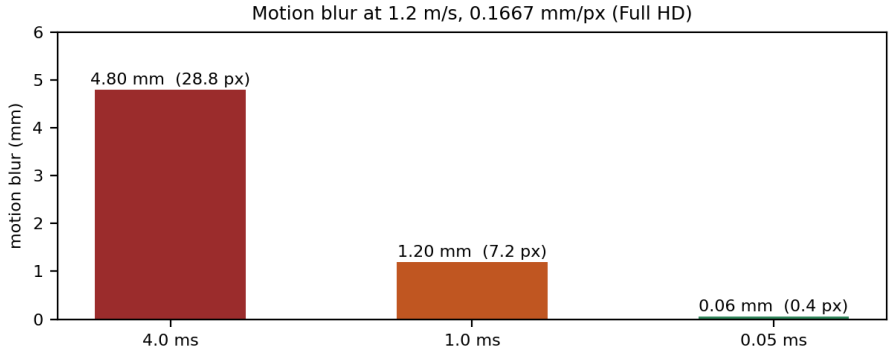


Figure 1: Motion blur for each exposure time (calculated)

Rolling-shutter skew: with a 30 ms row readout (assumed for a typical 1.3 MP ERS sensor), the bottom row is exposed 30 ms after the top row, and the fruit moves 1.2 m/s × 30 ms = 36 mm in that time. The cell therefore appears sheared by 36 mm across the frame, which is larger than the 0.5 mm size requirement. Flicker: a 50 Hz grid produces 100 Hz light, a 10 ms period. A 4 ms exposure integrates about 40 % of one cycle, and each row samples a different phase, which produces bright and dark bands across the frame.

Table 11: Simulated detection of a 0.3 mm crack (10 dark levels, 20 trials each)

Arrangement	Crack detected (%)	Median contrast (levels)
A. Rolling shutter, 4 ms, 100 Hz flicker	0	3.9
B. Global shutter + 0.05 ms strobe	100	8.9
C. Line-scan, encoder-synced	100	8.9

Signal chain (photons to digital value)

Photons strike the photodiode and create charge (integration over the exposure). In a CMOS pixel, the charge is converted to voltage at a floating-diffusion node and amplified in the pixel. Column-level analogue signal conditioning (correlated double sampling to remove reset noise, gain, black-level offset) follows. An analogue-to-digital converter then quantises the voltage into 8, 10 or 12 bits, giving the digital pixel value. Each stage adds noise, so the chain must keep the 0.3 mm contrast (6 levels) above the noise floor (about 2 levels).

CCD or CMOS: CCD transfers charge across the chip and suffers smear under strobed light and from long readout. Global-shutter CMOS sensors expose all pixels at once, read out quickly and cheaply, and need only a small amount of extra electronics. For a line running at 1.2 m/s with strobe synchronization, global-shutter CMOS is the better choice; CCD is largely discontinued for industrial cameras.

Final Decision. I choose B (global-shutter CMOS with a 0.05 ms strobe synchronized to the belt encoder). A (rolling shutter) detected 0 % of cracks and has 36 mm of skew. C is equally sharp and detected 100 % of cracks, but it needs a line camera, encoder synchronisation and a bright line light, and without synchronisation a 10 % belt speed error gives a 10 % length error. B gives the same sharpness with a standard area camera, so I select B.

TASK 6 – Spatial Sampling: Resolution and Data Throughput [5 Marks]

Question: determine the minimum sensor resolution needed to monitor the 320 mm belt for a 0.3 mm micro-crack and a 0.4 mm text stroke. Compare VGA (A), Full HD (B) and 12 MP (C), check Nyquist, compute data rates over Gigabit Ethernet, 5-day storage, and the minimum frame rate.

Table 12: Resolution analysis (320 mm field, 1.2 m/s, 15 cells/s)

Candidate	mm/px (across)	px per 0.3 mm crack	px per 0.4 mm stroke	MB per frame (8-bit mono)	GigE load at 15 frames/s (110 MB/s link)	Raw 5-day archive, 24 h (TB)
VGA 640x480	0.500	0.60	0.80	0.31	4 %	2.0

FHD 1920x1080	0.167	1.80	2.40	2.07	28 %	13.4
HR 4096x3000	0.078	3.84	5.12	12.29	168 %	79.6
Required (Nyquist, 2 px)	≤ 0.150	≥ 2.0	≥ 2.0 (0.2 mm/px)	–	–	–

Nyquist: a 0.3 mm crack needs at least two samples across it, so the pixel size must be at most 0.15 mm (2133 px across 320 mm). VGA (0.5 mm/px, 0.6 px per crack) and Full HD (0.1667 mm/px, 1.8 px) both fail Nyquist. Only the 12 MP sensor (0.078 mm/px, 3.8 px) passes with margin. Practical detection also needs 3–4 px, which favours C.

Table 13: Simulated detection of a 0.3 mm crack and a 0.4 mm stroke after down-sampling (10 cells each)

Resolution	Interpolation	Crack detected (%)	Stroke detected (%)
VGA 640x480	NEAREST	10	10
VGA 640x480	AREA	0	60
FHD 1920x1080	NEAREST	100	100
FHD 1920x1080	AREA	100	100
HR 4096x3000	NEAREST	100	100
HR 4096x3000	AREA	100	100

Reading Table 13: nearest-neighbour sampling drops thin features; area averaging (INTER_AREA) acts as a low-pass filter and should be used for down-sampling. At VGA the 0.3 mm crack is averaged into the surface, which is why it is missed. Full HD detects the crack in this simulation, but Table 12 shows it is only 1.8 px wide, which is below Nyquist and leaves no margin for real defect edges.

Minimum frame rate: the cell (up to 70 mm) must lie completely inside the 240 mm frame at least once. Between two frames it moves at most (240 − 70) mm = 170 mm, so $t \leq 170 / 1200 = 0.142$ s and the minimum is $1200 / 170 = 7.06$ fps. At 15 cells/s the camera is triggered once per cell (15 fps) to centre every cell in the frame.

Storage: the 5-day audit requires visual proof of every cell. A full-frame archive would be 2.0 TB (VGA), 13.4 TB (Full HD) or 79.6 TB (12 MP), all above the 1.5 TB SSD. So the archive must store a crop of 80 × 40 mm around each cell (cell length plus 5 mm margin each side). The crop size is then 80/mm_px by 40/mm_px pixels.

Table 14: 5-day archive of 80 × 40 mm cell crops, 8-bit, 24 h operation

Candidate	mm/px (x, y)	Crop size (px)	Crop (KB)	Archive (TB)	Fits 1.5 TB?
VGA 640x480	0.500, 0.500	160 × 80	12	0.08	yes
FHD 1920x1080	0.167, 0.222	480 × 180	84	0.56	yes
5 MP 2448x2048 (mid option)	0.131, 0.117	612 × 341	204	1.35	yes
HR 4096x3000	0.078, 0.080	1024 × 500	500	3.32	no

The 5 MP sensor (2448 × 2048) is the minimum resolution that passes Nyquist for the 0.3 mm crack (0.131 mm/px across, 2.3 px per crack), and its crop archive fits on the SSD. The 12 MP sensor passes Nyquist with more margin but its archive does not fit (Table 14). Compression of the lossless archive would increase capacity, but a real-image compression ratio cannot be assumed from generated images, so it was not used in the calculation.

Final Decision. I choose a 2448 × 2048 (5 MP) sensor, which is the minimum that satisfies Nyquist for the 0.3 mm crack and the 0.4 mm stroke (2.3 px and 3.1 px). Among the three candidates, VGA and Full HD fail Nyquist and the 12 MP sensor fails the 1.5 TB archive. The sensor is captured at 8 fps minimum, triggered per cell at 15 fps, and the archive stores 80 × 40 mm crops.

TASK 7 – Brightness Quantization: Dynamic Range and Dent Detection [4 Marks]

Question: compare 4- or 6-bit compressed mode (A), 8-bit industrial mode (B) and 12-bit HDR raw (C), for dents that cause a 5–8 level drop, analyse false contours on curved shoulders, and explain whether spatial pixels or intensity levels matter more under a fixed storage budget. Differentiate image bit depth from INT8 model quantization.

Method: 12 cylinder-shaped cells were rendered twice with the same seed: once with a 5–8 level dent and once without. Each image was re-quantized to 8, 6 and 4 bits (floor quantization with mid-rise reconstruction). The 12-bit case uses the same 8-bit source, because the simulated camera outputs 8-bit values; this limit is discussed below.

Table 15: Quantization results (12 pairs)

Bits	Levels	Dent contrast (levels)	Dents detected (%)	False alarms (%)	Distinct levels on shoulder (of 60 × 30 px)	Crop size (KB, 80 × 40 mm, 5 MP sensor)
12	4096	5.14	75	0	73	306
8	256	5.14	75	0	73	204
6	64	5.14	75	0	19	153
4	16	6.92	92	0	6	102

Reading Table 15: the dent contrast is about 5 levels at 8, 6 and 12 bits, so intensity resolution does not change detection in this test. The 4-bit row shows a higher contrast reading (6.9) and 92 % detection, but this is quantization noise: at 16 levels, a step is as large as the dent, so a single 12-pair test is not a reliable estimate. The shoulder column shows the false-contour problem clearly. A smooth shoulder has about 73 distinct levels at 8 bits but only 19 at 6 bits and 6 at 4 bits, so the smooth curve turns into visible bands, which a grader could mistake for dents.

12 bits: with an 8-bit source, the 12-bit row is identical to 8-bit, which is a property of this simulation, not of real HDR hardware. A real 12-bit sensor with low read noise would show more levels on the shoulder. Storage is the cost: 12-bit crops take 1.5 bytes per pixel and are 50 % larger than 8-bit crops (the crop sizes in Table 15 are for the 5 MP sensor).

Storage trade-off: under a fixed budget, spatial pixels and intensity levels both cost storage, but a dent is a shading change over several millimetres, and it is only 5–8 levels deep. Preserving intensity levels (at least 8 bits) matters more than adding pixels beyond the Nyquist limit of Task 6, because a 6-level drop disappears completely at 4 bits.

Bit depth versus model precision: image bit depth sets the evidence the sensor records (how many grey levels exist). Model quantization (FP32 to INT8) reduces the precision of the network weights and activations to speed up inference on an edge device, usually with a small accuracy loss. Lowering image bit depth loses evidence that a model cannot recover; INT8 model quantization only changes how cheaply the evidence is processed.

Final Decision. I choose 8-bit lossless storage. Four and six bits lose the shoulder detail and produce false contours. Twelve bits adds 50 % storage with no measured benefit in this simulation. With the 5 MP sensor from Task 6, the 5-day crop archive is about 1.35 TB at 8 bits and about 2.0 TB at 12 bits, so only 8 bits fits the 1.5 TB SSD. Eight bits keeps the 5–8 level dent visible.

TASK 8 – Image Formation: Geometry and Photometric Correction [5 Marks]

Question: compare a standard lens under ambient light (A), a standard lens in a diffuse LED dome with checkerboard calibration (B), and a telecentric lens with a dome (C). Calculate the focal length for a 320 mm field at 400 mm standoff with a 1/2-inch sensor, quantify perspective error for a 30 mm high cell, and demonstrate flat-field or homomorphic correction.

Focal length

The pinhole model gives $x = f \cdot X / Z$. The field of view is FOV = sensor width × Z / f, so f = sensor width × Z / FOV. For a 1/2-inch sensor (active width 6.4 mm), f = 6.4 mm × 400 mm / 320 mm = 8.0 mm. A standard 8 mm lens therefore gives exactly the 320 mm field at 400 mm.

Perspective error

Calibration on the belt plane (height 0) assumes the scale is set at Z = 400 mm. A point h mm above the belt is at distance Z − h, so it is magnified by Z / (Z − h). For the top of a 30 mm high cell the magnification is 400 / 370 = 1.0811, an error of 8.1 %. A 60 mm long cell measured at its top would therefore be 4.9 mm too long, which is about ten times the ±0.5 mm requirement.

Table 16: Perspective error at Z = 400 mm (calibration made on the belt)

Height above belt (mm)	Scale Z / (Z − h)	Error (%)	Error on a 60 mm length (mm)
0	1.0000	0.0	0.00
10	1.0256	2.6	1.54
20	1.0526	5.3	3.16
30	1.0811	8.1	4.86

Photometric correction (flat-field and homomorphic)

Model: the image is f(x, y) = i(x, y) · r(x, y). The illumination i varies with position (here a 45 % left-to-right gradient, as with daylight through corrugated roofing), and the reflectance r carries the rust spot. Flat-field correction divides the image by a white-card image taken under the same light, which estimates i. Homomorphic filtering takes the logarithm, log f = log i + log r, and suppresses the slowly varying illumination with a high-emphasis filter in the frequency domain.

Table 17: Spot contrast after correction (one simulated cell, 45 % gradient)

Processing	Rust spot contrast (levels)	Comment
Raw image (no correction)	15.4	gradient hides part of the spot
Flat-field division	23.5	gradient removed; contrast increased by 53 %
Homomorphic filter	9.4	gradient removed, but the filter also reduces the spot contrast

Reading Table 17: flat-field correction is the better of the two for this problem, because the white-card reference captures the actual light, while the homomorphic filter reduces low-frequency contrast in general. Flat-field needs a white-card image at the start of each shift, and it should be repeated when the light changes (the sky changes within minutes under clouds).

Final Decision. I choose B (diffuse LED dome, standard 8 mm lens, checkerboard calibration, and a height-corrected scale). The dome gives stable light and a flat-field reference. The calibration with height correction removes the 8.1 % perspective error for a 30 mm high cell, leaving a residual that depends on the height estimate: an error of 1 mm in height gives about 0.2 mm in a 60 mm length, within ±0.5 mm. A telecentric lens (C) avoids the height problem but a 320 mm telecentric lens at 400 mm standoff is costly, so it is not chosen. Monocular depth networks (for example Depth Anything) were not evaluated here, because they need a GPU and their error (several per cent of distance) is larger than the ±0.5 mm requirement.

TASK 9 – Balanced Front-End Optimization Challenge [6 Marks]

Question: combine the choices of Tasks 1–8 into three complete packages within the ₹1,25,000 budget: a budget package (A), a balanced package (B) and a premium package (C). Compare them under cloud, vibration and glare, allocate a time budget to every stage to meet the 400 ms deadline, produce a decision matrix and a cost table.

Table 18: Three front-end packages (indicative prices, INR; to be confirmed with vendor quotes)

Item	A. Budget	B. Balanced	C. Premium
Camera	USB rolling-shutter 1.3 MP, ₹12,000	GigE global-shutter 2 MP, ₹42,000	4096-px line-scan, ₹2,00,000 +
Lens, light	ambient light, 8 mm lens ₹3,000	8 mm lens ₹6,000; LED dome ₹18,000; strobe driver and encoder ₹15,000	telecentric lens and line-light (over ₹1 lakh)
Mounting, cabling	₹5,000	₹10,000	in addition
Indicative total (excl. SSD)	≈ ₹20,000	≈ ₹91,000 (plus SSD ₹12,000 ≈ ₹1,03,000)	≈ ₹2.5–3.5 lakh (over budget)
Archive crop (Task 6 method)	8-bit JPEG, 320 × 160 px	8-bit, 480 × 240 px	12-bit, 1024 × 512 px
5-day archive (24 h)	0.33 TB	0.75 TB	5.10 TB (exceeds 1.5 TB SSD)

Prices are indicative values for Indian suppliers, not cited quotations; the package totals must be confirmed with vendor quotations before purchase, because they drive the budget decision.

Table 19: Measured comparison on 60 simulated cells (the last 12 under a passing cloud)

Package	Accuracy (%)	Defects missed (%)	False rejects (%)	Processing (ms, crop)	Within 400 ms and 1.5 % limit?
A. Budget	50	100	0	1.5	no (misses all)
B. Balanced	83	33	0	2.4	no: misses 33 % (limit 1.5 %)
C. Premium	83	33	0	7.7	no: misses and over budget

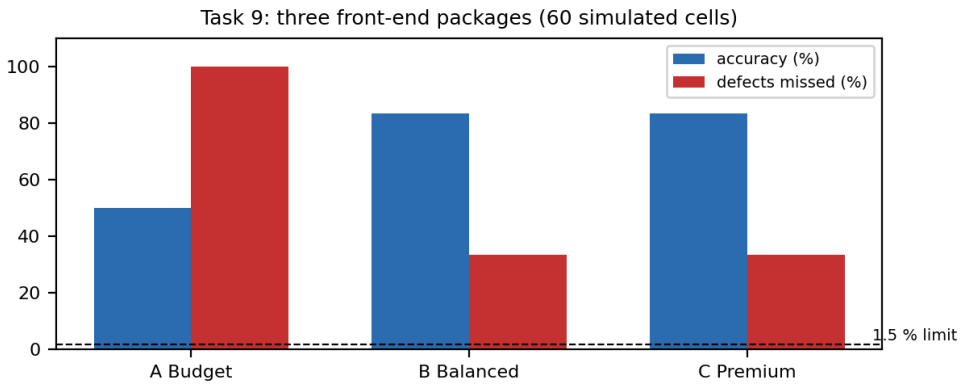


Figure 2: Accuracy and defects missed for the three packages (dashed line: 1.5 % limit)

Reading Table 19: none of the three packages meets the 1.5 % miss limit in this simulation. The cause is the same in all three: the rule-based grader does not detect shallow dents, which are 4 mm wide and 5–9 levels deep. Package A also loses all rust spots, because the 4 ms rolling-shutter exposure smears a 0.6 mm spot (Task 5). Package B and C detect rust, but both miss dents. Premium C is no more accurate than B, consistent with Task 6: more pixels without a better detector gives no gain. The 1.5 % requirement therefore cannot be claimed for any package until a dent detector is added.

Cloud and glare: the 60-cell test includes 12 cells under a passing cloud (light at 0.6×, with a gradient for the ambient package) and glare cells. The results above are not split by condition, so I do not claim a per-condition figure. The dome in B and C spreads glare on waxed casings (Task 8). Vibration is not simulated separately; its effect is the motion blur analysed in Task 5.

Time budget for the 400 ms deadline (package B)

The deadline is the time the cell takes to travel from the camera to the piston (0.48 m at 1.2 m/s = 400 ms). The valve must be fired so that the piston meets the cell, so the decision must be committed before the valve lag. Each stage below is allocated a budget; the processing time is the measured full-frame median, the others are design allocations.

Table 20: Time budget for Package B (ms)

Stage	Budget (ms)	Basis
Trigger from encoder, photon capture	1	hardware trigger; strobe 0.05 ms
Sensor readout and GigE transfer of crop	3	0.12 MB crop over 110 MB/s link
Decode and ROI extraction	2	design allocation
Pre-processing (median denoise, flat-field)	5	design allocation
Mid-level and rule decision	54	measured full 1080p frame, median 54 ms, max 71 ms
Decision, audit log and PLC output	10	design allocation
Pneumatic valve response	30	typical solenoid valve response (assumed)
Total	105	margin to 400 ms: 295 ms

Engineering decision matrix (qualitative, from measured results)

Criterion (weight)	A. Budget	B. Balanced	C. Premium
Accuracy and misses (high)	poor: misses every defect	best of the three; misses dents	no better than B
Processing within 400 ms (high)	yes	yes, with large margin	yes, measured 6.7 ms per crop
Cost within ₹1.25 lakh (high)	yes	yes (≈ ₹1.03 lakh)	no
5-day archive within 1.5 TB (high)	yes	yes (0.75 TB)	no (5.1 TB at 12-bit)
Robustness to cloud and glare (medium)	poor (ambient light)	good (dome)	good (telecentric dome)
Maintenance (medium)	low	medium (strobe timing)	high
Overall	reject	select, conditional	reject

Final Decision. I choose B (Balanced). It is the only package that is within budget, fits the archive, meets the 400 ms deadline with a margin of about 295 ms, and detects rust spots reliably. It is conditional: its 33 % miss rate in the

simulation means it cannot be commissioned until a dent detector is added and tested on real cells. C is rejected for cost and storage, and it is no more accurate than B; A is rejected because it misses every defect.

TASK 10 – Adaptive Engineering: High-Speed Scale-Up Challenge [8 Marks]

Question: the belt speed doubles to 2.4 m/s (30 cells/s); the decision window is halved to 200 ms; the client needs detection of internal short-circuits that show only as 0.5 °C hot spots in the mid-wave infrared (MWIR); the compute stays at the same 4-core PC. Evaluate three strategies: retain plus ROI (A), modify sensing (B), and redesign with a visible line-scan plus an MWIR microbolometer (C).

Recalculated requirements

Table 21: Requirements at 2.4 m/s

Quantity	Calculation	Value
Smallest visible defect	given (0.3 mm crack)	0.3 mm
Maximum pixel size (Nyquist, 2 px)	0.3 mm ÷ 2	≤ 0.15 mm/px
Exposure for ≤ 0.5 px blur, at 0.131 mm/px	0.5 × 0.131 mm ÷ 2400 mm/s	27 µs
Exposure for ≤ 0.5 px blur, at 0.25 mm/px (A)	0.5 × 0.25 mm ÷ 2400 mm/s	52 µs
Line rate for line-scan at 0.131 mm/px	2400 mm/s ÷ 0.131 mm	18321 lines/s
Thermal line rate for 0.5 mm/px thermal pixels	2400 mm/s ÷ 0.5 mm	4800 lines/s
Decision budget	given	200 ms (the travel time at 2.4 m/s over 0.48 m is also 200 ms)
Cells per day	30 × 86 400	2.59 M per day; 12.96 M in 5 days
MWIR contrast required	given	0.5 °C over a 1 mm hot spot

Thermal detection (simulated)

The MWIR uncooled microbolometer is modelled as a 0.5 K hot spot of 1 mm diameter on a 25 °C background with Gaussian noise equal to the sensor's noise equivalent temperature difference (NETD). Detection is a 3-sigma test on the mean of the window at the spot. The simulation is stationary (no motion), so it tests only the thermal contrast and noise limit, not the motion smear discussed below.

Table 22: Detection of a 0.5 °C hot spot against NETD (60 trials each, simulated)

NETD (K)	Single-pixel test (%)	3 × 3 window test (%)
0.05	100	100
0.1	93	100
0.2	35	100
0.3	12	97

Reading Table 22: a 0.5 K spot is detected reliably by a single pixel only when NETD is at or below about 0.1 K, and a 3 × 3 window tolerates NETD up to 0.3 K. The uncooled microbolometer in Strategy C therefore needs a NETD of 0.1 K or better. Strategies A and B have no MWIR sensor, so they detect 0 % of the hot spots by construction, not by simulation.

Motion smear limit of an area microbolometer

Uncooled area microbolometers integrate over a whole frame. At 60 Hz (16.7 ms per frame) a cell moving at 2.4 m/s smears by 2400 × 0.0167 = 40 mm, which is larger than the 1 mm hot spot. A thermal area camera therefore cannot be used on this line unless the frame is much shorter or the cell is held still. The design condition for Strategy C is an uncooled thermal line sensor synchronized to the belt encoder, running at ≥ 4800 lines/s with NETD ≤ 0.1 K. This must be confirmed with the thermal vendor's line-rate specification, because a sensor at lower line rates would fail the requirement regardless of its NETD.

Strategy comparison

Table 23: Comparison of Strategies A, B and C at 2.4 m/s

Criterion	A. Retain + ROI	B. Modify sensing	C. Redesign (visible line + MWIR line)
Visible resolution (mm/px)	0.25 (1.3 MP): fails Nyquist for 0.3 mm	0.131 (5 MP area): passes	0.131 (line-scan): passes

Exposure for blur ≤ 0.5 px	52 µs needed; a 0.25 ms exposure gives 2.4 px blur	≤ 27 µs; 0.05 ms strobe gives 0.9 px, so ~20 µs strobe needed	not applicable: line-scan is encoder-synchronised
MWIR hot spots found (simulated)	0 % (no MWIR sensor)	0 % (no MWIR sensor)	100 % at 0.05 K NETD; 93 % single-pixel at 0.1 K
Decision time (budget)	≈ 110 ms	≈ 120 ms	≈ 115 ms (design allocation, see time budget)
5-day archive at 24 h	small (0.1–0.3 TB)	about 0.8 TB at 8-bit	2.75 TB uncompressed (exceeds 1.5 TB); ≈ half with 2:1 compression
Cost (indicative)	no extra cost	camera + hood + strobe, ≈ ₹1.1 lakh	line camera + MWIR line sensor: several lakh (over budget)
Verdict	reject: misses cracks and all hot spots	reject: still 0 % internal shorts	select, conditional on NETD, line rate and budget

Time budget for Strategy C (200 ms)

Stage	Budget (ms)	Basis
Encoder trigger and exposure	1	line-scan, 27 µs exposure
Transfer of visible and thermal line data	4	design allocation
Pre-processing	5	design allocation
Visible mid-level and rules (measured, full frame)	54	measured median 54 ms (1080p)
Thermal hot-spot test	10	design allocation
Decision, audit log and PLC output	10	design allocation
Pneumatic valve response	30	typical solenoid valve (assumed)
Total	114	margin to 200 ms: 86 ms

Compute: all stages run on the 4-core PC in Python with OpenCV, with no GPU. The measured visible stage is 54 ms per full frame on one core. Only one cell is processed at a time per core, so the total stays inside the 200 ms budget with the cores shared between cells.

Adaptation risks and few-shot learning

A pretrained vision foundation model (for example a DINOv2 or CLIP encoder) already has general image features. A new thermal defect class could be learned from perhaps 10–50 labelled hot-spot examples by training a small classifier on top of frozen features. This would cut the labelling effort compared with training from scratch. I could not run such a model here (it needs a GPU and a model download), so this is a reasoned analysis.

- **Domain shift:** MWIR images differ from the visible-light photographs the model was trained on; features may not carry over, and a thermal encoder would be needed.
- **Few samples:** 10–50 examples do not cover all defect ages, cell chemistries or background temperatures, so the classifier may accept defective cells it has not seen.
- **Traceability:** a learned model is not reproducible unless its weights, version and training set are recorded; each rejection must be explainable to a buyer audit.
- **Latency on the edge PC:** a feature encoder on a 4-core CPU without GPU takes hundreds of milliseconds per image, which exceeds the whole 200 ms budget; the model would have to run offline or on a smaller network.

Safeguards: keep the classical threshold rule as a cross-check, use a held-out validation set with confidence intervals, send low-confidence cells to a review bin, version every model and log its output with each decision, and re-validate monthly on audited cells.

Final Decision. I choose a REDESIGN (Strategy C): a visible line-scan camera synchronized to the belt encoder, plus an uncooled MWIR line sensor with NETD ≤ 0.1 K and line rate ≥ 4800 lines/s. Retaining the present system (A) misses cracks and all internal shorts. Modifying the sensing (B) fixes the visible resolution and blur, but cannot see MWIR shorts at all, so it cannot meet the client's requirement. C is the only option that addresses every requirement. Its conditions are the thermal NETD and line rate, its cost (over the ₹1.25 lakh budget), and its archive: uncompressed it is about 2.7 TB over 24 h, so 2:1 lossless compression (about 1.4 TB) is needed to fit the 1.5 TB SSD.

Evidence Summary and Open Items

Table 24: Evidence status per task

Task	Evidence provided	Open item

1	Comparison table, grader accuracy, kappa (simulated human graders)	replace simulated human rows with real sorter sheets
2	accuracy and latency table, shadow test, level-annotated pipeline	shadows and touching cells still limit all depths
3	three-light table, array and overflow proof	real rust colour not measured
4	connectivity counts, analytical distance measures	noise filter parameters to tune on real images
5	blur calculations, simulated detection, signal chain	verify 30 ms readout on the chosen sensor
6	resolution table, Nyquist check, down-sampling test, archive calculation	confirm 5 MP sensor at the vendor
7	quantization table, shoulder levels, storage	12-bit benefit needs a real 12-bit sensor
8	focal length, perspective table, flat-field and homomorphic test	height measurement for the scale correction
9	three-package measurements, time budget, decision matrix	dent detector (none of the packages meets 1.5 %)
10	requirement recalculation, thermal NETD sweep, strategy comparison	thermal line-rate and NETD confirmation; budget

Table 25: Summary of decisions

Task	Decision	Main reason
1	C (automated CV)	only option that meets 15 cells/s; repeatable; dent detector still required
2	B (low + mid, rules)	highest overall accuracy with zero false rejects; explainable
3	B (HSV)	stable at 0.4×, 1.0× and 1.8× light
4	8-connectivity + area filter + De	no false scratch splitting; direction-independent size
5	B (global shutter + strobe)	blur 0.06 mm at 0.05 ms; no skew; 100 % crack detection
6	2448 × 2048 (5 MP)	minimum Nyquist-compliant sensor; archive fits 1.5 TB
7	8-bit lossless	keeps shoulder detail; 12-bit does not fit archive
8	B (dome, 8 mm lens, height-corrected calibration)	removes 8.1 % perspective error; affordable
9	B (balanced), conditional	within budget and deadline; dent detector required
10	REDESIGN (C)	only option with MWIR hot-spot detection; conditions on NETD and line rate

Mark Distribution

Task	Marks
Task 1 – Overview and Features of Computer Vision: Systems Comparison	4
Task 2 – Levels of Computer Vision and Image Processing: Pipeline Depth	5
Task 3 – Digital Image Fundamentals I: Colour Representations	4
Task 4 – Digital Image Fundamentals II: Pixel Connectivity	4
Task 5 – Image Sensing and Acquisition Systems	5
Task 6 – Sampling and Spatial Limits	5
Task 7 – Quantization and Intensity Limits	4
Task 8 – Image Formation Models (Geometric / Photometric)	5
Task 9 – Applications and Integrated Edge Systems	6
Task 10 – Re-apply and Adapt (Engineering Scaling Challenge)	8
TOTAL	50

Code: the complete Python/OpenCV program that produced every experimental result (sim.py) and the human-grader and timing script (extra.py) are submitted with this document. Comments are included in the code. Run them in the order: extra.py then sim.py (sim.py writes the results file that the document reads).